Technical Report — Case Study 1

Title: AUTOSAR HLD AI Assistant: Grounded AI for High-Level Design Analysis, Engineering Q&A and Architecture Exploration

University: [TO BE FILLED] Student: [TO BE FILLED] Register No.: [TO BE FILLED]

Report status: Academic draft. No newly measured Gemini benchmark or signed declaration is asserted.

1. Problem statement and users

AUTOSAR-like High-Level Design (HLD) documents describe software components, provided/required ports, interfaces and architectural data flow. Information is spread across pages and tables. Manual reading is slow; simple search lacks paraphrase tolerance; unrestricted LLMs can fabricate precise engineering facts. Intended users are automotive software engineers, architecture reviewers, systems/integration engineers and academic evaluators. The goal is grounded document Q&A plus structured architecture exploration, with human oversight.

2. Input data and knowledge base

Text-based HLD PDF files enter via upload. The repository contains synthetic Braking System HLD (../data/samples/AUTOSAR_HLD_Sample_BrakingSystem.pdf) and ten-question golden dataset (../data/golden_dataset.json) (seven answerable, three unanswerable). A separate Door Control HLD was manually tested but is not checked into this repo. ChromaDB stores embedded chunks and evidence metadata; SQLite stores SWC/Port/Interface entities. Runtime databases are regenerated from inputs, not model-training data.

3. Architecture / workflow

Ingestion: Streamlit → FastAPI → coordinator → PyMuPDF page/block extraction → cleaning → hierarchical chunking → all-MiniLM-L6-v2 embeddings → ChromaDB, plus deterministic entity extraction → Pydantic validation/deduplication → SQLite.

Question answering: user query → embedding → semantic top-k retrieval → evidence-delimited grounded prompt → MockLLM or Gemini → citation ID verification → Streamlit answer, citations, page/section/snippet, grounding warnings.

Architecture browsing: Streamlit → FastAPI entities endpoint → SQLite → SWC/Port/Interface tables. See Mermaid architecture (ARCHITECTURE_AND_DATA_FLOW.md).

4. Implementation and configuration

The project is Python-based with separate core, backend, frontend, storage, config and tests packages. PyMuPDF preserves block and page metadata; TextCleaner handles repeated headers/footers, hyphenation and whitespace; HierarchicalChunker produces provenance-linked text chunks.

The pretrained sentence transformer all-MiniLM-L6-v2 converts chunk/query text to semantic vectors. ChromaDB returns nearest chunks, ordered by similarity. The normal coordinator uses

top-k 5; its relevance cutoff defaults to None. GroundedPromptBuilder limits context to 12,000 chars, wraps document text in untrusted <context> / <chunk> regions, requests [chunk_id] citations and requires explicit insufficient evidence for absent information.

BaseLLMProvider enables deterministic MockLLM and a Gemini provider. The coordinator supplies generation temperature 0.0. Actual Gemini model is environment-dependent: config/settings.py defaults to gemini-3-flash-preview, while the tracked .env.example selects gemini-2.5-flash. Actual run settings need independent confirmation. Never commit the actual key.

CitationVerifier rejects unknown cited IDs, resolves known IDs to document/page/section/snippet and labels explicit insufficient-evidence responses. It does not semantically prove the answer. This distinction is necessary because the earlier MockLLM yielded generic cited outputs even when a specific factual response or negative refusal was expected.

EntityExtractor identifies SWCs, Ports and Interfaces using patterns and table parsing, validates entities and deduplicates before SQLite persistence. Optional LLM fallback exists, but the upload coordinator explicitly sets use_llm=False, so default ingestion is rules-based. FastAPI exposes GET /health, POST /documents/upload, POST /rag/query and GET /entities/{doc_id}; Streamlit supplies the workbench.

5. Evaluation steps and results

The repository includes tests covering configuration, PDF parsing/cleaning, chunking, embeddings, Chroma, retrieval, prompt builder, mock/provider integration, citations, extraction, evaluation, FastAPI and UI. The golden dataset contains 10 questions; PipelineEvaluator (../core/evaluation/evaluator.py) calculates retrieval success, citation-ID validity, groundedness flags, negative-grounding and keyword-overlap ratios. These are implementation-defined proxies and must not be represented as human-verified factual accuracy.

A student earlier reported 70 passed, 1 skipped (historical) and 16 indexed chunks on a Door Control test (historical). Fresh automated tests were not executed as part of this documentation commit. The tested MockLLM produced a generic cited answer to an unsupported MCU query, so a passing negative-grounding test is not established. Protocol and status (../Evaluation_Results/EVALUATION_STATUS.md) document remaining evidence.

6. Responsible AI, privacy and security

Evidence-restricted prompting; citation-ID checks with human source review; explicit abstention requirement; untrusted-document delimiter handling; configuration via private .env; avoiding confidential input PDFs in public GitHub; student responsibility for technical conclusions. Safety limits: valid IDs do not entail factual claims, PDF prompt-injection defenses are partial, externally hosted providers may receive engineering content, and no compliance/ASIL claim has been established. This software is advisory.

7. Innovation highlights

1. Traceable source chunks with page/section/snippet.

2. Deterministic citation-ID validation and warnings.

3. AUTOSAR-specific SWC/Port/Interface extraction.

4. Dual databases: ChromaDB semantic index and SQLite structured entities.

5. Modular local MockLLM / Gemini providers with testability.

6. End-to-end Streamlit + FastAPI workbench rather than raw LLM chat.

8. Limitations and future scope

Text-only PDF assumptions; irregular tables/diagrams; exact identifier retrieval; no OCR/ARXML in core path; no authenticated tenant isolation; citation validity not semantic entailment. Future enhancements: hybrid BM25+vector retrieval, graph relationships, ARXML, measured benchmark coverage, entailment testing, secured enterprise deployment.

9. Conclusion

The project demonstrates domain-specific integration of pretrained ML embeddings and a generative LLM with evidence retrieval, provenance validation and structured automotive architecture extraction. It is a working prototype intended to assist, not replace, human engineering review.

10. Referenced sources

• Project code: https://github.com/SABA718/AUTOSAR-HLD-AI-Assistant

• AUTOSAR: https://www.autosar.org/

• PyMuPDF: https://pymupdf.readthedocs.io/

• Sentence Transformers: https://www.sbert.net/

• Chroma: https://docs.trychroma.com/

• Gemini: https://ai.google.dev/gemini-api/docs

• FastAPI: https://fastapi.tiangolo.com/

• Streamlit: https://docs.streamlit.io/